Automated QA for the Eclypse platform

A robot that uses the platform like a client's team does, proves every action really worked, never touches client data, and explains failures in plain English. The prototype runs it against a stand-in platform with one command: `npm run qa`.

1. How it works, when it runs, what it checks

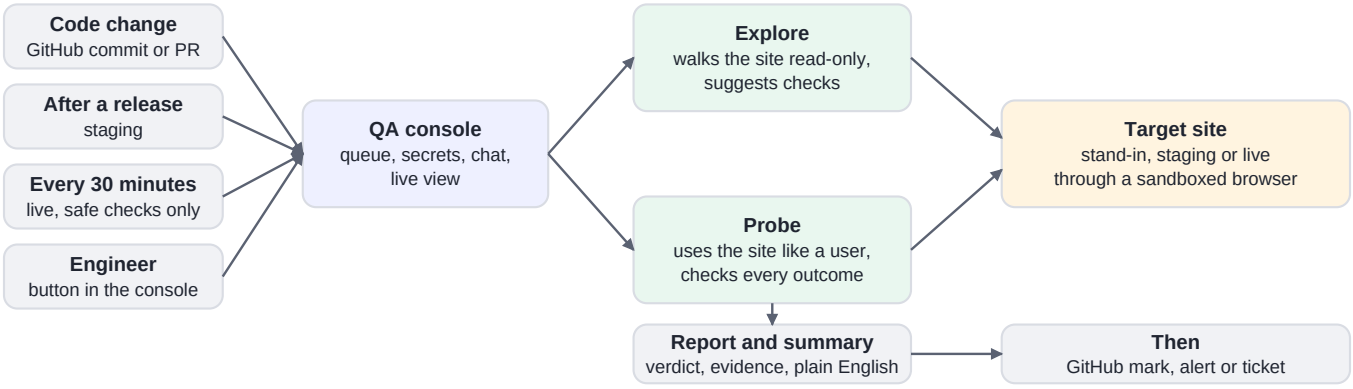


Figure 1. Any trigger goes through the QA console, which drives a sandboxed browser against the target and reports back.

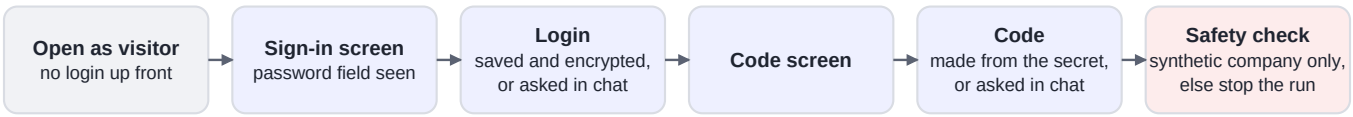
When	Target	What runs	On failure
Every code change	Fresh build of the branch	Full Probe, AI mocked (free, true facts)	Blocks the merge, marks the commit
After every release	Staging	Full Probe, real AI answers replayed	Halts the release
Every 30 minutes	Live, synthetic company	Read-only safe checks	Pages only after two failures in a row
Every 6 hours	Live, synthetic company	Safe checks plus one small real AI job	As above

What it checks: outcomes, not page loads. The uploaded file appears with the right row count and survives a reload. Delete removes that document and nothing else. A saved setting is still there after a reload. An AI job reaches Complete and every number it reports matches the input file. Downloads match the screen. Wrong password, wrong code and a bad file each fail with a clear message. A button that does nothing when clicked is a failure.

2. Tools, and why

Choice	Why	Not chosen
Playwright (TypeScript)	Real browsers, waits automatically (the main cure for flaky tests), handles uploads and downloads, can block network requests (the safety guard), records traces. Same language as the frontend.	Cypress: weaker on downloads and multi-site. Selenium: more plumbing, more flake.
Find controls by role and label	Tests click what a user sees, so a redesign does not break them, and a missing label is a finding in itself.	CSS selectors break on every redesign.
Authenticator codes (TOTP)	The robot computes the same code a phone shows. No backdoor in the product.	Email codes: slow, flaky inbox polling.
Claude Haiku, kept on a leash	Explores new sites, plans journeys once, writes summaries. Cheap and fast. Never decides pass or fail.	AI in the verdict: not repeatable, costs every run.
GitHub	Runs on every change. Works even as a collaborator, by watching the branch and marking each commit.	Webhooks and apps need repo admin rights.

3. Signing in safely when sign-in needs a one-time code



Passwords and codes go straight to the browser. They never reach Claude, the chat history or a log.

Figure 2. The robot signs in only when it reaches a sign-in screen, and stops if it lands anywhere but the synthetic company.

- **A real user with real two-step sign-in.** The QA account's authenticator secret lives in a secret store per environment; the robot computes the six-digit code from it. Nothing in the product skips the code for tests.
- **A code works once.** The robot remembers the last time window it used, waits for a fresh one, and never uses the last two seconds of a window, so a slow page cannot fail on an expiring code.

- **One sign-in per run,** session reused, runs on the same account never overlap: no lockouts.
- **When nobody saved a login,** the run pauses and asks in the job chat. The answer goes straight to the browser.

4. Staying safe on the live platform

The rule: the robot can only see and change synthetic data, and it stops if it cannot prove that.

1. **Least privilege.** The QA account belongs to one synthetic company. Client accounts are created with random logins that are thrown away, so the robot cannot sign in as a client even if told to.
2. **Fail closed.** After sign-in it checks which company it is in, and stops the whole run if it is not the synthetic one.
3. **Write allow-list in the browser.** Every request that would change data is checked. Only expected ones pass, and a delete is only allowed for something this test created. On any site not on the allow-list, nothing that saves or deletes is sent at all.
4. **Canary instead of client data.** A second synthetic company holds records with a unique marker. Seeing that marker anywhere, in any test, is a critical failure. Fetching its document by exact id must return 404.
5. **Evidence stays clean.** Password and code fields are blacked out in screenshots and the live view. Production keeps only the text report. No page content goes to an AI model in the verdict path.

5. AI output that changes on every run, and cost

Check facts, ignore wording. The robot uploads a synthetic file whose correct answers were worked out by hand. Every number the AI reports must equal them, the summary must exist and contain no junk, and downloads must match the screen. The prose is never pass or fail: odd numbers in it only raise a warning.

Long jobs. The robot watches what the user sees and fails early and specifically, for example "stuck at Analyzing with AI, 75%, no change for 15 seconds", not after a vague ten-minute timeout.

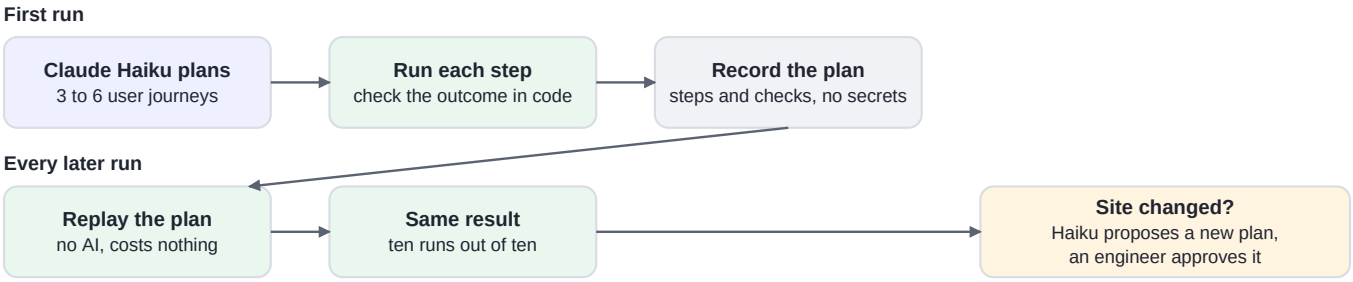


Figure 3. On a new site, Claude plans once. The recorded plan then replays without AI, so results repeat and cost nothing.

Where	AI	Cost
Every code change	Mocked	Nothing
Release runs	Real model, recorded once, replayed by a hash of model, prompt and input	Close to nothing per rerun
Live, a few times a day	Real Claude Haiku on the smallest file	Well under a dollar a month
Explore, first Probe on a new site	Haiku with a step cap, token cap and cache; cost shown on the job	Cents per run

6. What happens when it finds a problem

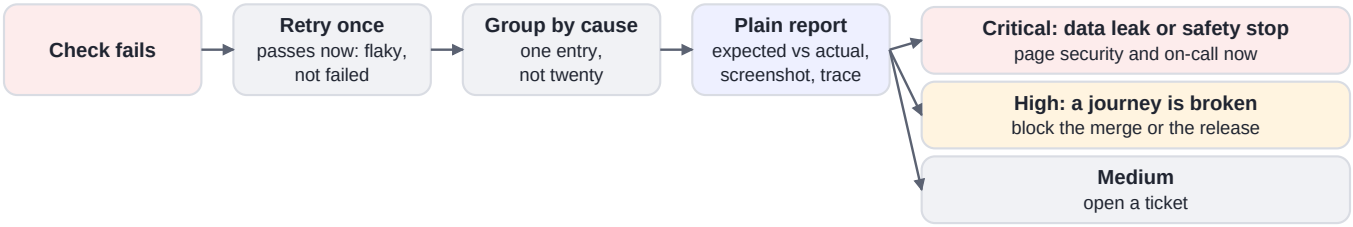


Figure 4. One clear entry per problem, routed by how serious it is.

- **No false alarms.** An unreachable environment is one result, not twenty failures. Live checks must fail twice before anyone is paged. Browser console noise is evidence, not a failure. A slow but working page must pass, and the bug matrix proves it does.
- **Easy to understand.** Every job ends with a short summary written by Claude from the scrubbed log, plus the live recording of what the robot did. The summary never changes the verdict.

7. Risks the brief did not mention, and results

- Robot traffic polluting analytics and billing: requests are tagged and the account is known.
- An AI provider outage looking like a product bug: reported as an environment problem.
- Prompt injection from a site Claude explores: page text is treated as data, and the browser guard blocks every write whatever the model chooses.
- Canary deleted, so isolation tests pass without proving anything: a daily check that it still exists (next step).

Command	Result
`npm run qa`	PASS: 22 checks, about 60 seconds including a 20 second AI job.
`npm run qa:bugs`	10 of 10 injected bugs caught, each with a specific headline; the slow page passes.
`npm run qa:repeat`	10 runs with nothing changed, all identical, though the mock AI words its summary differently each time.

Built with Claude Code as a pair programmer; every decision and result was reviewed and run locally.